

ÜBUNG 9

24WS-UE09-A01 Point - Punkt

Implementieren Sie eine Klasse `Point`, die einen Punkt mit x- und y-Koordinate repräsentiert. Erstellen Sie zwei Konstruktoren und Methoden zum Auslesen und Setzen der Attributwerte (get/set-Methoden). Hinweis: Achten Sie jeweils auf sinnvolle Sichtbarkeitsattribute sowie auf gültige Parameterwerte und verwenden Sie ggf. selbstgewählte sinnvolle Ersatzwerte!

Die Verwendung der Klasse `Point` sieht folgendermaßen aus:

```
public class PointTest{  
    public static void main(String[] args) {  
        Point p1 = new Point();  
        Point p2 = new Point(1.5, 3.5);  
        double distance = p1.distance(p2);  
        Out.println("Distance from " + p1.toString() + " to " + p2 + " = " + distance);  
    }  
}
```

Das ergibt folgende Ausgabe:

```
Distance from (0.0/0.0) to (1.5/3.5) = 3.81
```

Implementieren Sie folgende Funktionalität für die Klasse `Point`:

- Konstruktor 1: parameterlos, initialisiert den Punkt mit der x- und y-Koordinate jeweils 0.0 (ruft Konstruktor 2 auf)
- Konstruktor 2: nimmt die x- und y-Koordinate entgegen
- get-Methoden für die x- und y-Koordinate: `getX`, `getY`
- set-Methoden für die x- und y-Koordinate: `setX`, `setY`
- `distance(Point p2)`: berechnet den Abstand des aktuellen Punktes zum angegebenen Punkt im Parameter gerundet auf 2 Kommastellen
 - Der Abstand zwischen 2 Punkten `p1` und `p2` ist die Quadratwurzel aus $(p2.x - p1.x)^2 + (p2.y - p1.y)^2$
 - Sie können die Funktion `Math.sqrt(...)` zum Berechnen der Quadratwurzel verwenden
 - Sie können die Funktion `Math.round(val * 100.0)/100.0` zum Runden verwenden
- `toString()`: textuelle Repräsentation des Punktes (siehe Beispiel Ausgabe)

Zeigen Sie anhand dreier sinnvoller Testfälle in einer Klasse `PointTest`, dass Ihr Programm funktioniert!

24WS-UE09-A02 Line - Linie

Eine Linie besteht aus Startpunkt und Endpunkt. Die Klasse `Line` wird gemeinsam mit der Klasse `Point` (aus Aufgabe 1) verwendet. Kopieren Sie dazu Ihre Datei `Point.java` aus Aufgabe 1 in den Ordner `exercise`. Implementieren Sie anschließend die Klasse `Line` mit zwei Punkten zur Verwaltung von Startpunkt und Endpunkt der Linie. Hinweis: Achten Sie jeweils auf sinnvolle Sichtbarkeitsattribute sowie auf gültige Parameterwerte und verwenden Sie ggf. selbstgewählte sinnvolle Ersatzwerte!

Die Verwendung der Klasse `Line` sieht folgendermaßen aus:

```
public class LineTest{  
    public static void main(String[] args) {  
        Line l1 = new Line (2.0, 1.0, 3.0, 2.0);  
        Line l2 = new Line (new Point(2.0, 0.0), new Point(4.0, 2.0));  
        Out.println("Line 1 from " + l1.getStartpoint() + " to " + l1.getEndpoint()  
            + ", gradient: " + l1.calculateGradient());  
        Out.println("Line 2 from " + l2.getStartpoint() + " to " + l2.getEndpoint()  
            + ", gradient: " + l2.calculateGradient());  
        Out.println("parallel: " + l1.isParallel(l2));  
    }  
}
```

Das ergibt folgende Ausgabe:

```
Line 1 from (2.0/1.0) to (3.0/2.0), gradient: 1.0  
Line 2 from (2.0/0.0) to (4.0/2.0), gradient: 1.0  
parallel: true
```

Implementieren Sie die Klasse `Line` nach folgendem Muster:

```
public class Line {  
    //Instanzvariablen  
    ...  
    //Konstruktor 1: erhält x- und y-Koordinate des Startpunkts sowie x- und y-Koordinate des Endpunkts  
    //(ruft Konstruktor 2 auf)  
    ...  
    //Konstruktor 2: erhält Start- und Endpunkt  
    ...  
    //getXXX: get-Methoden für Startpunkt und Endpunkt (getStartpoint, getEndpoint)
```

```
...  
    //calculateGradient: ermittelt die Steigung der Line, die wie folgt berechnet wird: (endY-startY)/(endX-startX)  
    ...  
    //isParallel: ermittelt ob die aktuelle Linie zur im Parameter angegebenen Linie parallel verläuft.  
    //Parallel bedeutet, dass beide Linien die gleiche Steigung haben.  
    //Achtung: double-Werte nicht auf Gleichheit prüfen, sondern mit epsilon 0.0001.  
    //Math.abs() und Math.round() dürfen verwendet werden.  
    ...  
}
```

Zeigen Sie außerdem anhand dreier sinnvoller Testfälle in einer Klasse `LineTest`, dass Ihr Programm funktioniert!

24WS-UE09-A03 Fussballstadion - Stadium

Implementieren Sie eine Klasse `Stadium`, die ein Fußballstadion mit Namen (`name`), Stadt (`city`), Kapazität (`capacity`) und das Eröffnungsjahr (`openingYear`) repräsentiert. Erstellen Sie drei Konstruktoren und Methoden zur Manipulation der Instanzvariablen. Hinweis: Achten Sie jeweils auf sinnvolle Sichtbarkeitsattribute sowie auf gültige Parameterwerte und verwenden Sie ggf. selbstgewählte sinnvolle Ersatzwerte!

Die Verwendung der Klasse `Stadium` sieht folgendermaßen aus:

```
public class StadiumTest{  
    public static void main(String[] args) {  
        Stadium lusail = new Stadium("Lusail Iconic Stadium", "Lusail", 88966, 2022);  
        Stadium alBayt = new Stadium("Al Bayt Stadium");  
        alBayt.setCity("Al Khor");  
        alBayt.setCapacity(68895);  
        alBayt.setOpeningYear(2021);  
        Stadium alThumama = new Stadium("Al Thumama Stadium", "Doha", 44400, 2021);  
        Stadium stadium974 = new Stadium("Stadium 974", "Doha", 44089, 2021);  
        Out.println(lusail.toString());  
        Out.println(alBayt);  
        Out.println(alThumama);  
        Out.println(stadium974);  
    }  
}
```

Das ergibt folgende Ausgabe:

```
Lusail Iconic Stadium (City: Lusail, Capacity: 88966, Opening: 2022);  
Al Bayt Stadium (City: Al Khor, Capacity: 68895, Opening: 2021);  
Al Thumama Stadium (City: Doha, Capacity:44400, Opening: 2021);  
Stadium 974 (City: Doha, Capacity: 44089, Opening: 2021);
```

Implementieren Sie die Klasse `Stadium` nach folgendem Muster:

- Konstruktor 1: nimmt Namen, Stadt, Kapazität und das Eröffnungsjahr als Parameter entgegen
- Konstruktor 2: nimmt Namen, Stadt, Kapazität als Parameter entgegen, ruft Konstruktor 1 auf (Eröffnungsjahr wird auf 0 gesetzt)
- Konstruktor 3: nimmt den Namen als Parameter entgegen und ruft Konstruktor 1 auf (Stadt wird auf "" gesetzt, Kapazität und Eröffnungsjahr auf 0)

- get-Methoden für alle Instanzvariablen: getName, getCity, getCapacity, getOpeningYear
- set-Methoden für alle Instanzvariablen: setName, setCity, setCapacity, setOpeningYear
- toString: textuelle Repräsentation (siehe Beispiel Ausgabe)

Zeigen Sie anhand dreier sinnvoller Testfälle in einer Klasse StadiumTest, dass Ihr Programm funktioniert!

24WS-UE09-A04 Fussballturniere - Tournament

Im Jahr 2022 findet die Fußballweltmeisterschaft in Wüstenstaat Qatar in mehreren Stadien statt. Implementieren Sie dazu die Klasse `WorldCup` mit einem Feld von `Stadium`-Objekten, um die Stadien einer Weltmeisterschaft zu verwalten. Sie sollen dazu die Klassen `Stadium` von Aufgabe 2 wiederverwenden. Fügen Sie dazu die Klasse im Ordner `exercise` ein. Hinweis: Achten Sie jeweils auf sinnvolle Sichtbarkeitsattribute sowie auf gültige Parameterwerte und verwenden Sie ggf. selbstgewählte sinnvolle Ersatzwerte!

Die Verwendung der Klasse `WorldCup` sieht folgendermaßen aus:

```
public class WorldCupTest {  
    public static void main(String[] args) {  
        WorldCup qatar2022 = new WorldCup("Qatar", 2022, 8);  
        Stadium lusail = new Stadium("Lusail Iconic Stadium", "Lusail", 88966, 2022);  
        qatar2022.addStadium(lusail);  
        qatar2022.addStadium(new Stadium("Al Bayt Stadium", "Al Khor", 68895, 2021));  
        qatar2022.addStadium(new Stadium("Al Thumama Stadium", "Doha", 44400, 2021));  
        qatar2022.addStadium(new Stadium("Stadium 974", "Doha", 44089, 2021));  
        Out.println(qatar2022);  
        Out.println("total capacity: " + qatar2022.totalCapacity());  
    }  
}
```

Das ergibt folgende Ausgabe:

```
Qatar 2022 (4 of 8 stadiums specified)  
total capacity: 246350
```

Implementieren Sie die Klasse `WorldCup` nach folgendem Muster:

- Konstruktor: erhält den Namen des Gastgeberlandes, das Austragungsjahr und die maximale Anzahl an Stadien
- `getHostCountry()`: liefert den Namen des Gastgeberlandes
- `getYear()`: liefert das Austragungsjahr
- `addStadium(Stadium st)`: Stadium hinzufügen
- `containsStadium(String stadiumName)`: überprüft, ob in der Weltmeisterschaft bereits ein Stadium mit dem Namen im Parameter existiert und liefert bei einem Treffer `true`, ansonsten `false`
- `totalCapacity()`: ermittelt die Gesamtkapazität aller Stadien
- `toString()`: textuelle Repräsentation (siehe Beispiel Ausgabe)

Zeigen Sie anhand dreier sinnvoller Testfälle in einer Klasse `WorldCupTest`, dass Ihr Programm funktioniert!

24WS-UE09-A05 Sudoku Validierung und Ausgabe

Erweitern Sie Ihre Klasse `Sudoku` und implementieren Sie Schritt für Schritt noch folgende Methoden:

- `public void setField(int[][] field)`: prüft die im Parameter angegebene Sudoku-Matrix auf null-Werte und Feldgrößen (Zeilen, Spalten). Wenn die Matrix im Parameter gültig ist, dann soll sie die interne Matrix ersetzen.
- `public boolean areAllColumnsValid`: prüft alle Spalten auf Gültigkeit und liefert als Ergebnis `true` oder `false` zurück. Eine Sudoku-Spalte ist gültig, wenn sie alle Ziffern zwischen 1 und 9 genau einmal enthält.
- `public boolean areAllBoxesValid`: prüft alle Boxen (3x3) auf Gültigkeit und liefert als Ergebnis `true` oder `false` zurück. Eine Box ist gültig, wenn sie alle Ziffern zwischen 1 und 9 genau einmal enthält.
- `public String toString()`: gibt die String-Representation der Sudoku-Matrix zurück. Fehler in den Zeilen, Spalten und Boxen sollen mit einem Stern `*` davor und dannach hervorgehoben werden, siehe Beispiel mit der Datei `invalidValues.txt`:

```
+-----+-----+-----+
|*1**3* 4| 6 7 8| 9 *1**3*|
| 6 7 2| 1 9 5|*3* 4 8|
|*1* 9 8| 3 4 2| 5 6 7|
+-----+-----+-----+
| 8 5 9| 7 6 *1*| 4 *1**3*|
| 4 2 6| 8 5 3| 7 9 *1*|
| 7 1 3| 9 2 4| 8 5 6|
+-----+-----+-----+
| 9 6 1| 5 3 7| 2 8 4|
| 2 8 7| 4 1 9| 6 3 5|
| 3 4 5| 2 8 6| 1 7 9|
+-----+-----+-----+
```

Testen Sie Ihre Klasse `Sudoku` in der `main`-Methode Ihrer `Main`-Klasse. Erzeugen Sie ein Objekt der Klasse `Sudoku` und testen Sie mit den Dateien `validValues.txt` und `invalidValues.txt`! Vergessen Sie nicht, auch die Fehlerfälle zu überprüfen.